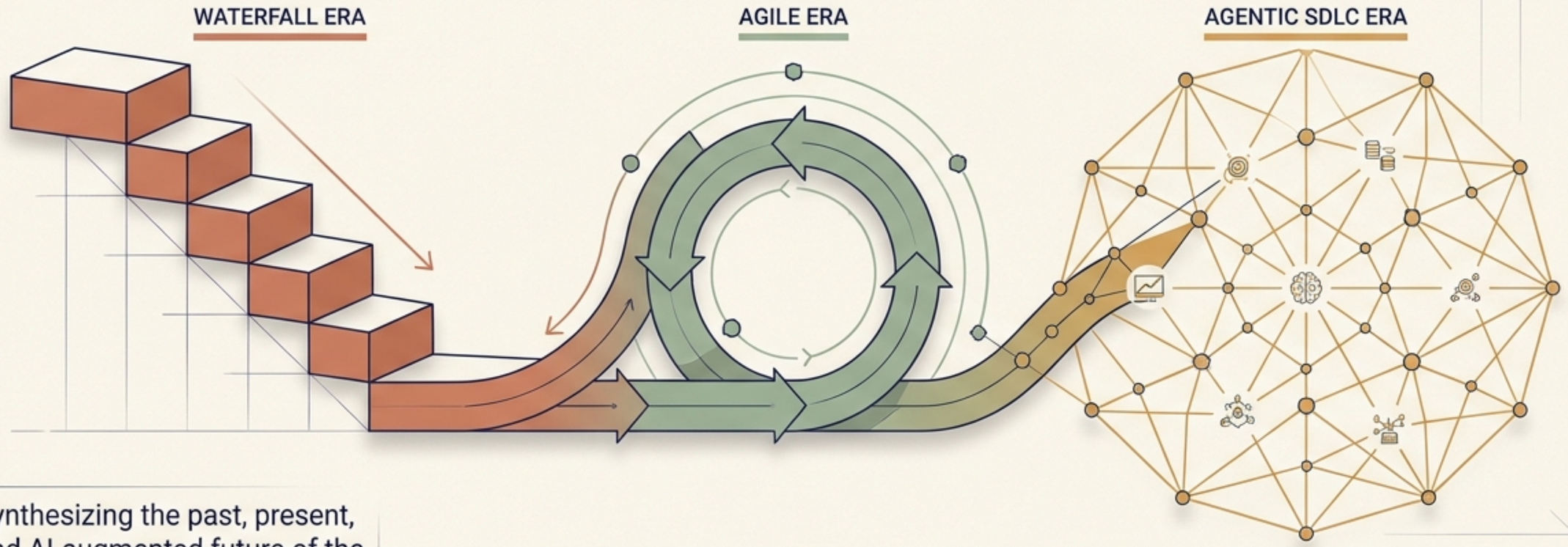
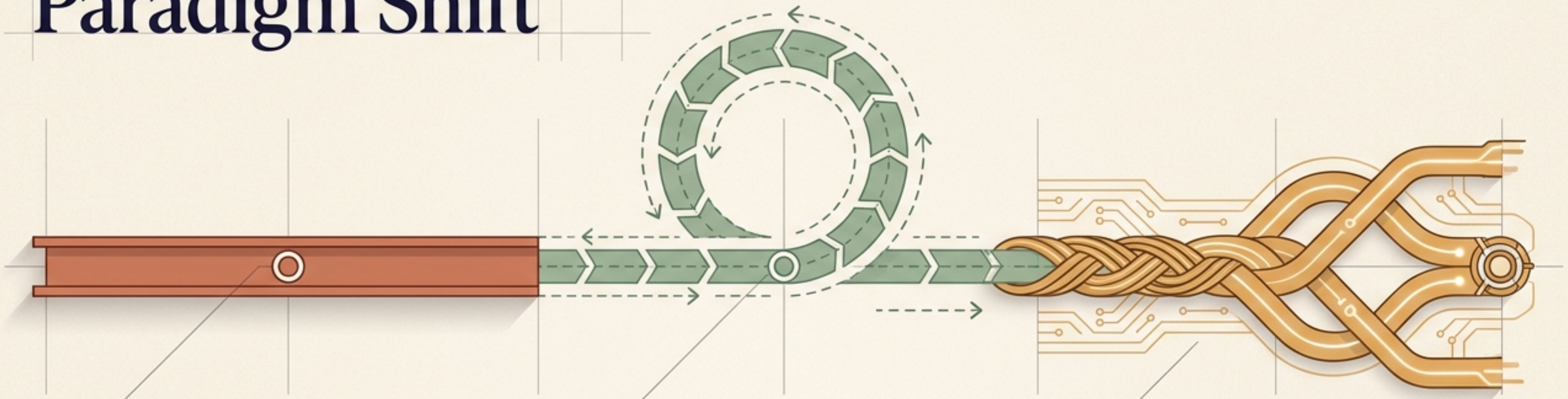
The Evolution of Software Development

From Waterfall to the Agentic SDLC



Synthesizing the past, present, and AI-augmented future of the software lifecycle.

The Timeline of a Paradigm Shift



1970s: The Software Crisis & Waterfall

Goal: Predictability

Managing the chaos of early software through rigid, sequential planning.

2001: The Agile Manifesto & Scrum

Goal: Adaptability

Shifting from process-heavy documentation to iterative, people-centric cycles.

2024+: Large Language Models & AI

Goal: Autonomy

Integrating reasoning AI agents to automate and enhance complex engineering tasks.

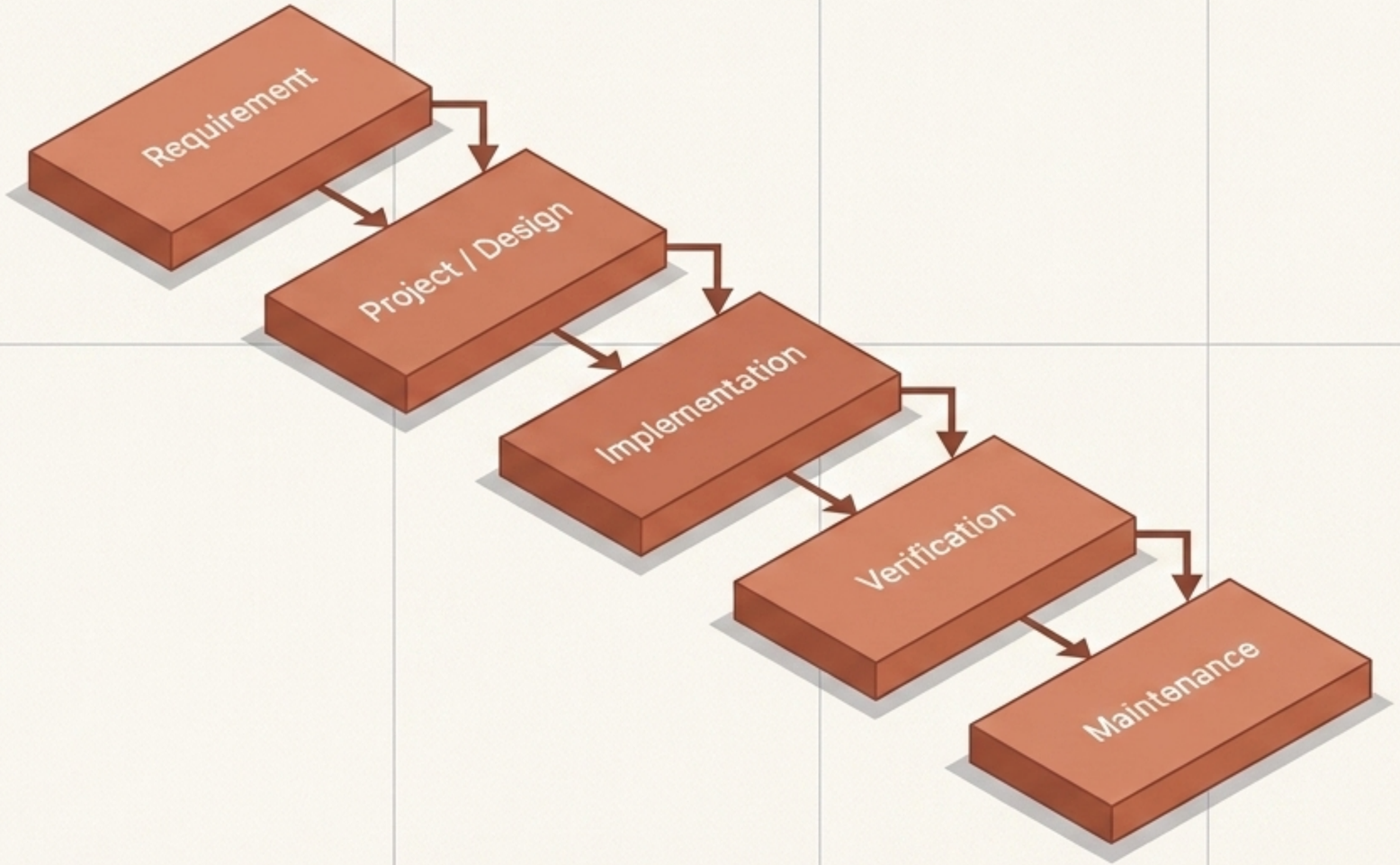
The Traditional Baseline: Waterfall

Core Philosophy

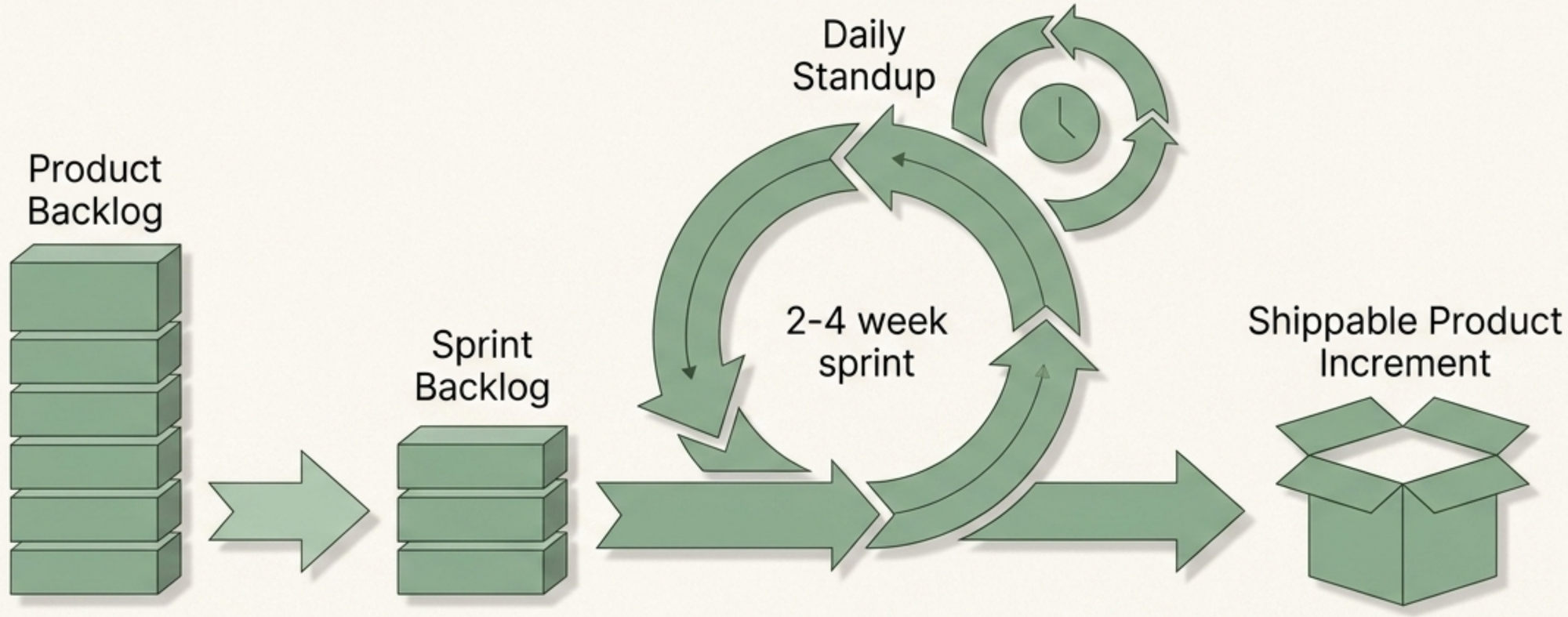
Heavy upfront planning and comprehensive documentation. Design is completely finalized before coding begins.

The Fatal Flaw

The customer is involved at the Requirement stage but does not see the product until Verification. Months of silence separate design from delivery, creating extreme vulnerability to shifting market needs.



The Agile Revolution: Scrum



The Shift	Key Drivers	Core Roles
From process-heavy, rigid planning to people-heavy, iterative cycles.	Face-to-face communication, adaptive planning, and rapid, continuous feedback.	Product Owner (maximizes value), Scrum Master (removes impediments), and Dev Team (executes).

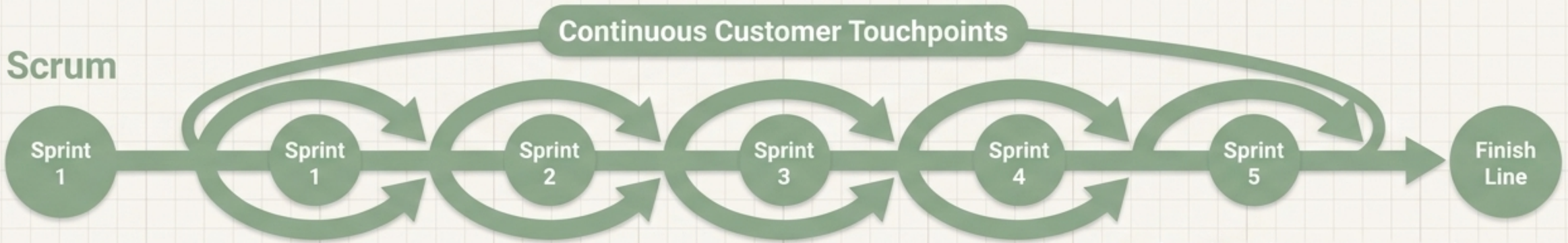
Case Study: Building a Chat App

Empirical comparison of methodologies (Oliveira et al.)

Waterfall

Result: Delivered a working chat app, heavily documented, but entirely rigid.

Scrum



Result: Delivered a superior app with emergent features born from continuous feedback (e.g., exclusive webmail login, instant camera attachments, contact groups).

Redefining Requirements Engineering (RE)

Traditional RE

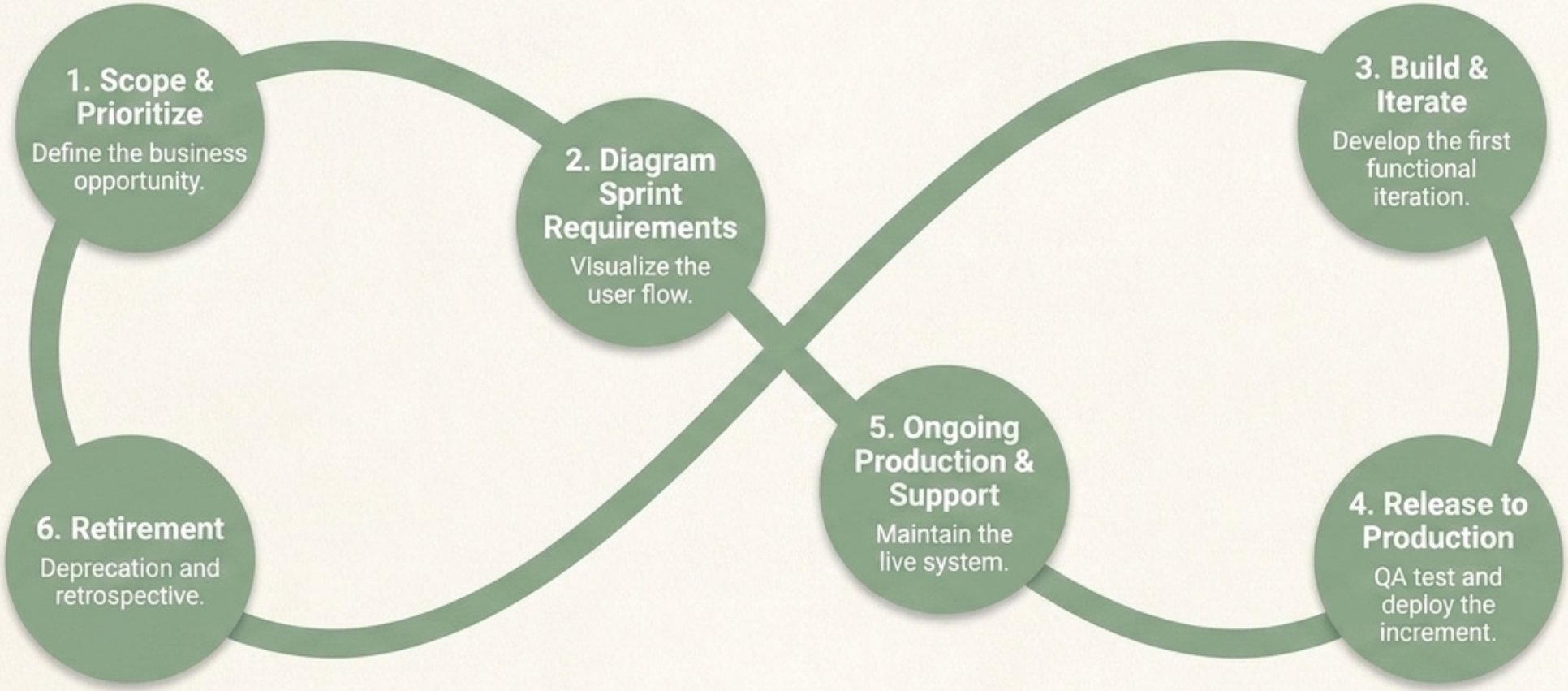
- Focus on comprehensive, upfront specifications.
- Systems Analyst operates as a siloed role.
- Relies heavily on rigid Use Case documents.
- The goal is to create documentation.

The fundamental goal transitions from creating documentation to share knowledge to fostering face-to-face communication to achieve a shared understanding.

Agile RE

- Focus on living backlogs and continuous refinement.
- Product Owner drives the overarching vision.
- Captures intent via simple User Stories.
- The goal is to foster conversation.

The Agile Software Development Life Cycle



This entire infinity loop executes in micro-cycles (Sprints of 2 to 4 weeks), not over years.

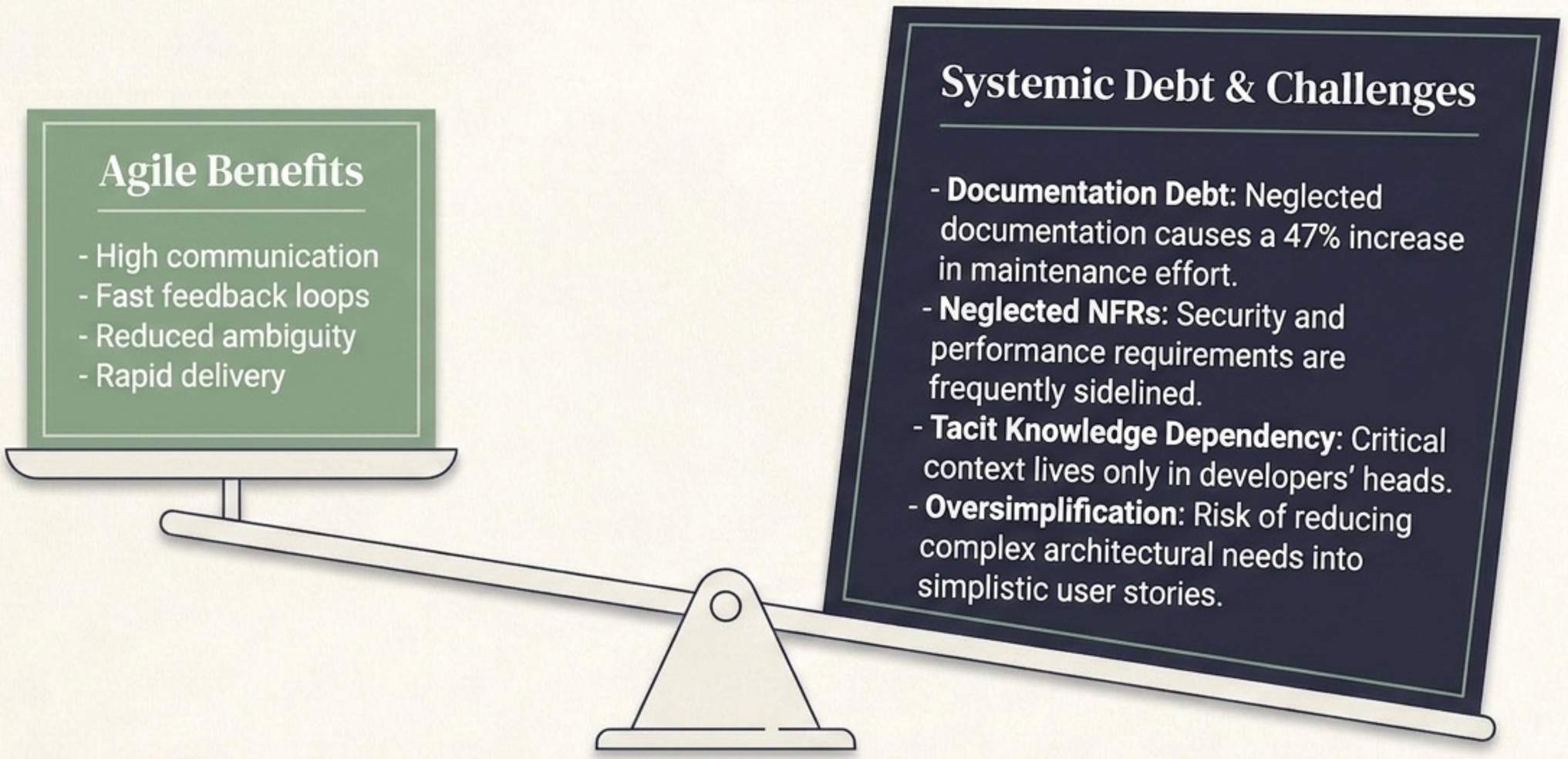
Agile Elicitation & Analysis in Practice

Frequency of techniques cited in empirical literature (Fraga & Barbosa)

The Ultimate Metric

Requirements in Agile are ruthlessly prioritized and ordered based on one driving factor: Business Value.

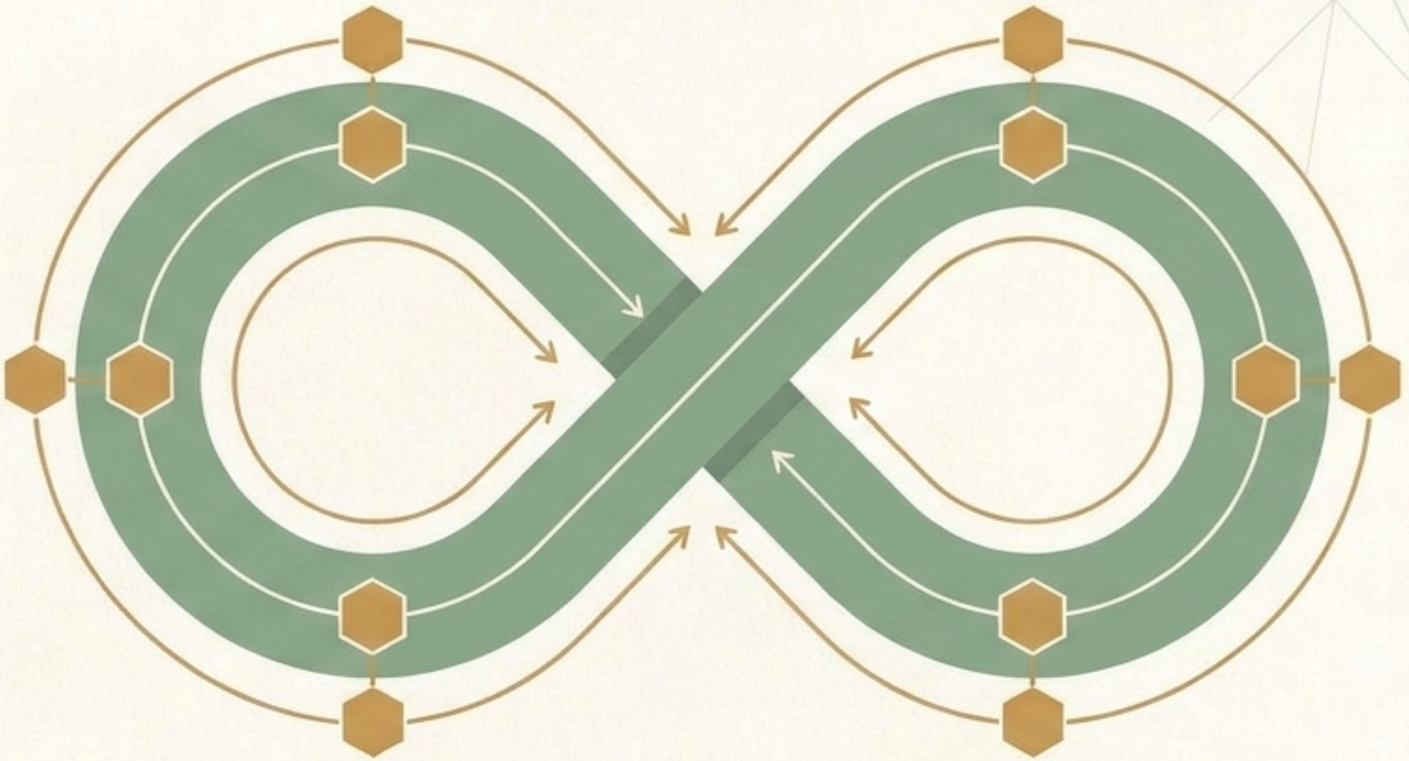
The Double-Edged Sword of Agile



The Next Frontier: Agentic SDLC

The Agentic SDLC (2024+)

An emerging paradigm where AI agents and Large Language Models (LLMs) are deeply integrated into the lifecycle.



Agents do not merely assist via autocomplete. They automate, accelerate, and enhance end-to-end engineering tasks that historically demanded intensive, manual human effort—solving the systemic bottlenecks of Agile.

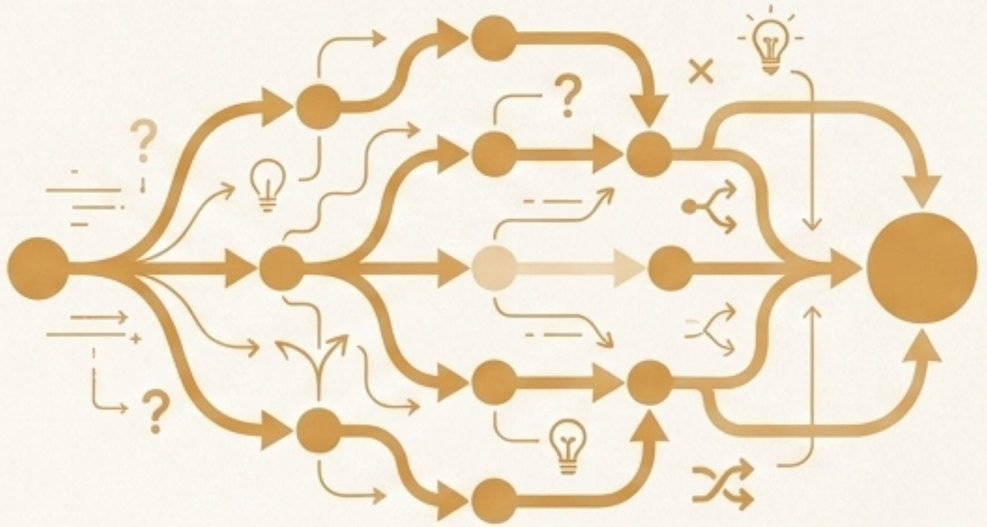
From Scripts to Reasoning

Pre-2022: DevOps/CI-CD

Deterministic Automation

- Humans write strict scripts.
- Machines blindly execute them.
- If X, then Y.
- Fails when encountering ambiguity or missing parameters.

Post-2024: Agentic AI



Probabilistic Reasoning

- Agents understand natural language.
- AI reasons through ambiguous requirements.
- Autonomously plans sequences of actions.
- Tools like GitHub Copilot, Devin, and Cursor act as cognitive collaborators.

AI Integration Across the SDLC

The era of the GenAI Teammate.

Requirements & Planning

AI automatically generates detailed user stories from rough meeting transcripts and audits backlogs to catch neglected Non-Functional Requirements (NFRs).

System Design

Agents translate natural language requirements directly into precise UML, architectural diagrams, and schema definitions.

Implementation & QA

Autonomous code generation, self-healing syntax correction during build failures, and automated QA test generation spanning edge cases.

Bridging the Gaps in Agile

How Agentic AI Solves Persistent Agile Pain Points.

Problem: Crippling Documentation Debt	Solution: Automated, real-time codebase documentation generated dynamically as code is committed.
Problem: Neglected Non-Functional Requirements	Solution: AI agents act as governance guardrails, automatically enforcing security and performance standards during sprint planning.
Problem: Tacit Knowledge Loss	Solution: LLMs act as conversational, semantic repositories of project history, queryable by any new team member.

The Master Comparison Matrix

	Traditional (Waterfall)	Current (Agile Scrum)	Future (Agentic SDLC)
Core Philosophy	Predictive	Adaptive	Autonomous
Requirements Format	Heavy Specs (BRDs)	User Stories & Backlogs	Dynamic Prompts & Semantic Context
Customer Touchpoints	Start and Finish	Every 2-4 Weeks (Sprints)	Continuous & Simulated via AI Proxies
Automation Level	Minimal	Deterministic (CI/CD Scripts)	Probabilistic (Reasoning Agents)
Primary Bottleneck	Changing Requirements	Developer Bandwidth & Doc Debt	AI Context Windows & Trust

The Future is Collaborativ **Collaborative**

"Agility in the 21st century requires the face-to-face empathy of human product owners combined with the autonomous, tireless execution of AI agents."

The organizations that master this hybrid SDLC will not just build software faster—they will define the next era of digital innovation.